

Deep Learning Lab 5

Suraj Karki
Department of Engineering Science
University West
Trollhättan, Sweden
suraj.karki@student.hv.se

Nils Karlsson
Department of Engineering Science
University West
Trollhättan, Sweden
nils.Karlsson@student.hv.se

Nils Wikström
Department of Engineering Science
University West
Trollhättan, Sweden
nils.wikstrom@student.hv.se

Index Terms—Deep Learning, Laboration 5, CNN

Abstract—The source code for this lab can be found at [1].

I. TASK 1

A. Apply convolution on a custom image and show the results.

First, we constructed a custom image of size 70×120 with three colour channels (RGB). The image was extracted from a sample dataset, centre-cropped to the required spatial dimensions, and rescaled to the interval $[0, 1]$ for numerical stability during convolution.

Figure 1 shows the custom images after preprocessing.



Fig. 1: Custom image 1 after preprocessing.

Then we applied convolution layer with 32 filters and a kernel size of 7×7 on the custom images. Figure 2 shows the result of applying convolution on custom images. The convolution layer was able to extract features from the images, which can be used for further processing in a neural network.

B. Apply max-pooling, depth-wise pooling, and average pooling on the images from the previous step and show the results.

We applied max-pooling, depth-wise pooling, and average pooling on the convolution result from the previous step. Figure 3, Figure 4, and Figure 5 shows the first feature map of applying different pooling techniques to the convolution result. Max-pooling takes the maximum value from each patch of the feature map, while average pooling takes the average value. Depth-wise pooling is a type of pooling that operates on each channel separately. The pooling layers help to reduce the spatial dimensions of the feature maps while retaining important features, which can improve the performance of the neural network.

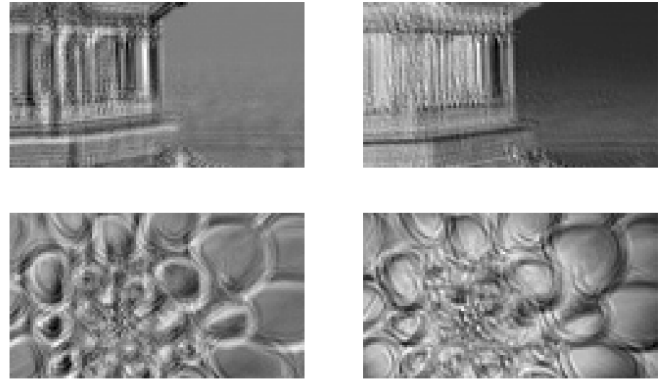


Fig. 2: Result of applying convolution on custom images.

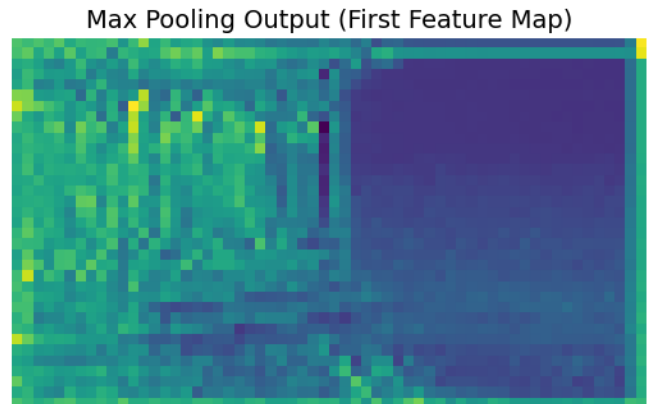


Fig. 3: Result of applying max-pooling on the convolution result.

II. TASK 2- CREATE AND TRAIN A CONVOLUTIONAL NEURAL NETWORK FOR THE MNIST FASHION DATASET AND COMPARE THE RESULTS OF THE CNN AND FCN.

In previous lab, a Fully Connected Network (FCN) was implemented for the classification of the Fashion MNIST dataset. While FCNs treat images as flattened vectors and ignore spatial structure, Convolutional Neural Networks (CNNs) explicitly exploit local spatial correlations through convolutional filters and pooling operations. This typically leads to improved generalisation performance and greater parameter efficiency.

Although FCNs are capable of learning non-linear deci-

Average Pooling Output (First Feature Map)

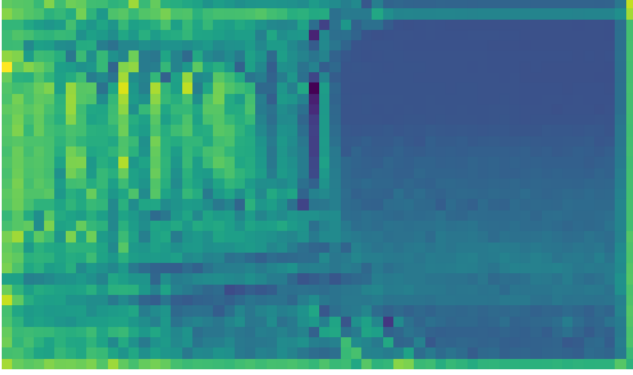


Fig. 4: Result of applying average pooling on the convolution result.

Depthwise Convolution Output (First Feature Map)

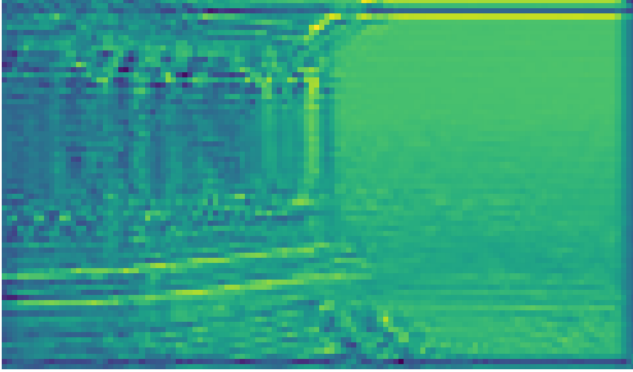


Fig. 5: Result of applying depth-wise pooling on the convolution result.

sion boundaries, they treat input images as flattened vectors, thereby discarding spatial structure. In contrast, CNNs leverage convolutional filters to capture local spatial features such as edges and textures, and pooling layers to achieve translation invariance.

The CNN consists of multiple convolutional layers organised into hierarchical blocks. Each convolutional layer applies learnable filters that extract local spatial features such as edges, textures, and object parts. ReLU activation functions introduce non-linearity, while He initialisation ensures stable gradient propagation during training.

Max pooling layers are inserted between convolutional blocks to reduce spatial dimensions. This downsampling operation provides translation invariance and reduces computational complexity. As the network depth increases, the number of filters is increased, allowing the model to capture progressively more abstract representations.

After the convolutional feature extraction stage, the feature maps are flattened and passed through fully connected layers. Dropout regularisation is applied to mitigate overfitting by randomly deactivating neurons during training. Finally, a

softmax output layer produces probability distributions over the ten Fashion MNIST classes. Figure 6 illustrates the result from single conv layer CNN.

For further experimentation, another cnn architecture was implemented with more convolutional layers combining the FCN. The results of the second CNN architecture are shown in Figure 7. The second CNN architecture achieved over 99% accuracy, demonstrating the importance of architectural design.

- **Convolutional Layers:** Two convolutional layers were used:

- 1) The first layer applies 32 filters of size 3×3 with same padding, followed by a ReLU activation.
- 2) The second layer applies 64 filters of size 3×3 , also with same padding and ReLU activation.

These layers extract low- to mid-level spatial features such as edges, textures, and simple patterns.

- **Max Pooling:** To reduce the spatial dimensions of the feature maps.
- **Flattening:** The multi-dimensional feature maps are flattened into a 1D vector, preparing the data for fully connected layers.
- **Fully Connected Layers and Dropout:** The first dense layer has 128 units with ReLU activation, followed by a dropout of 25% to reduce overfitting. The second dense layer acts as the output layer with 10 units corresponding to the 10 Fashion MNIST classes and a softmax activation to produce probability distributions. An additional dropout of 50% is applied before the output.

III. TASK 3- IMPLEMENT A RESNET-34 CNN MODEL USING KERAS

ResNet (Residual Network) introduces identity-based skip connections to address the degradation problem observed in very deep neural networks [2]. Instead of learning a direct mapping $H(x)$, each residual block learns a residual function:

$$F(x) = H(x) - x$$

The output of a residual unit is therefore defined as:

$$y = F(x) + x$$

This resnet facilitates gradient flow during backpropagation and mitigates vanishing gradient issues, enabling the successful training of substantially deeper architectures [2]. Each residual unit consists of two convolutional layers with 3×3 kernels, followed by batch normalisation. A ReLU activation function is applied after the first convolution and after the residual addition.

The structure of the main branch is:

$$\text{Conv} \rightarrow \text{BatchNorm} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{BatchNorm}$$

When spatial dimensions or channel depth change (i.e. when stride > 1), a 1×1 convolution is applied in the skip

```

Epoch 1/10
... 1719/1719 ----- 27s 11ms/step - accuracy: 0.6255 - loss: 1.0507 - val_accuracy: 0.8754 - val_loss: 0.3562
Epoch 2/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.8577 - loss: 0.4319 - val_accuracy: 0.8930 - val_loss: 0.2978
Epoch 3/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.8823 - loss: 0.3520 - val_accuracy: 0.8984 - val_loss: 0.2741
Epoch 4/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.8955 - loss: 0.3124 - val_accuracy: 0.9016 - val_loss: 0.2730
Epoch 5/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.9008 - loss: 0.2964 - val_accuracy: 0.9040 - val_loss: 0.2688
Epoch 6/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.9095 - loss: 0.2722 - val_accuracy: 0.8956 - val_loss: 0.2942
Epoch 7/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.9146 - loss: 0.2510 - val_accuracy: 0.9106 - val_loss: 0.2818
Epoch 8/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.9213 - loss: 0.2308 - val_accuracy: 0.9088 - val_loss: 0.2690
Epoch 9/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.9218 - loss: 0.2289 - val_accuracy: 0.9044 - val_loss: 0.2997
Epoch 10/10
1719/1719 ----- 14s 8ms/step - accuracy: 0.9276 - loss: 0.2103 - val_accuracy: 0.9078 - val_loss: 0.2846
313/313 ----- 2s 6ms/step - accuracy: 0.8962 - loss: 0.3561
1/1 ----- 1s 720ms/step

```

Fig. 6: Result of training the CNN on Fashion MNIST dataset.

```

... Epoch 1/10
1719/1719 ----- 15s 6ms/step - accuracy: 0.8854 - loss: 0.3756 - val_accuracy: 0.9864 - val_loss: 0.0444
Epoch 2/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9736 - loss: 0.0896 - val_accuracy: 0.9878 - val_loss: 0.0427
Epoch 3/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9811 - loss: 0.0631 - val_accuracy: 0.9896 - val_loss: 0.0345
Epoch 4/10
1719/1719 ----- 8s 5ms/step - accuracy: 0.9842 - loss: 0.0507 - val_accuracy: 0.9900 - val_loss: 0.0375
Epoch 5/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9878 - loss: 0.0404 - val_accuracy: 0.9920 - val_loss: 0.0352
Epoch 6/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9881 - loss: 0.0364 - val_accuracy: 0.9922 - val_loss: 0.0378
Epoch 7/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9891 - loss: 0.0305 - val_accuracy: 0.9920 - val_loss: 0.0364
Epoch 8/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9909 - loss: 0.0304 - val_accuracy: 0.9922 - val_loss: 0.0375
Epoch 9/10
1719/1719 ----- 7s 4ms/step - accuracy: 0.9933 - loss: 0.0222 - val_accuracy: 0.9924 - val_loss: 0.0361
Epoch 10/10
1719/1719 ----- 10s 4ms/step - accuracy: 0.9926 - loss: 0.0227 - val_accuracy: 0.9918 - val_loss: 0.0392
313/313 ----- 1s 4ms/step - accuracy: 0.9902 - loss: 0.0355
[0.034455474466085434, 0.991599977016449]

```

Fig. 7: Result of training the second CNN architecture on Fashion MNIST dataset (2 conv layer).

branch to match dimensions before addition. Otherwise, the skip connection acts as an identity mapping.

He initialisation is used for all convolutional layers to ensure stable variance propagation in deep networks. The model follows the canonical ResNet-34 configuration:

$$[64] \times 3, \quad [128] \times 4, \quad [256] \times 6, \quad [512] \times 3$$

Summary of the architecture:

- Initial 7×7 convolution with stride 2
- Batch normalisation and ReLU activation
- 3×3 max pooling
- 16 residual units organised in four stages
- Global average pooling
- Fully connected softmax layer with 10 outputs

Global average pooling replaces traditional dense layers, significantly reducing the number of parameters and improving generalisation. The model was trained on the Fashion MNIST dataset. The network was compiled using:

- Loss function: Sparse categorical cross-entropy
- Optimiser: Nadam
- Evaluation metric: Accuracy

Training was conducted for 10 epochs using a validation split. Model performance was subsequently evaluated on the held-out test set.

The ResNet-34 architecture achieved high classification accuracy on the Fashion MNIST dataset. Compared to earlier experiments: Figure 8 and Figure 9 demonstrates the training and validation accuracy for the ResNet-34 model.

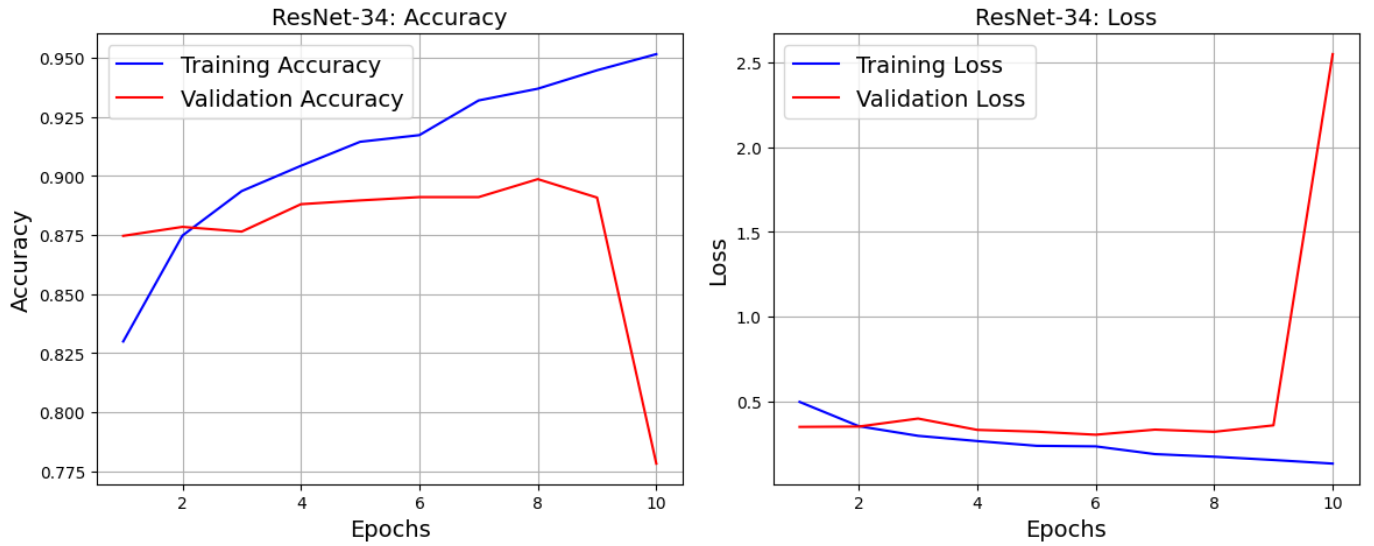


Fig. 8: Training and validation accuracy for ResNet-34 model on Fashion MNIST dataset.

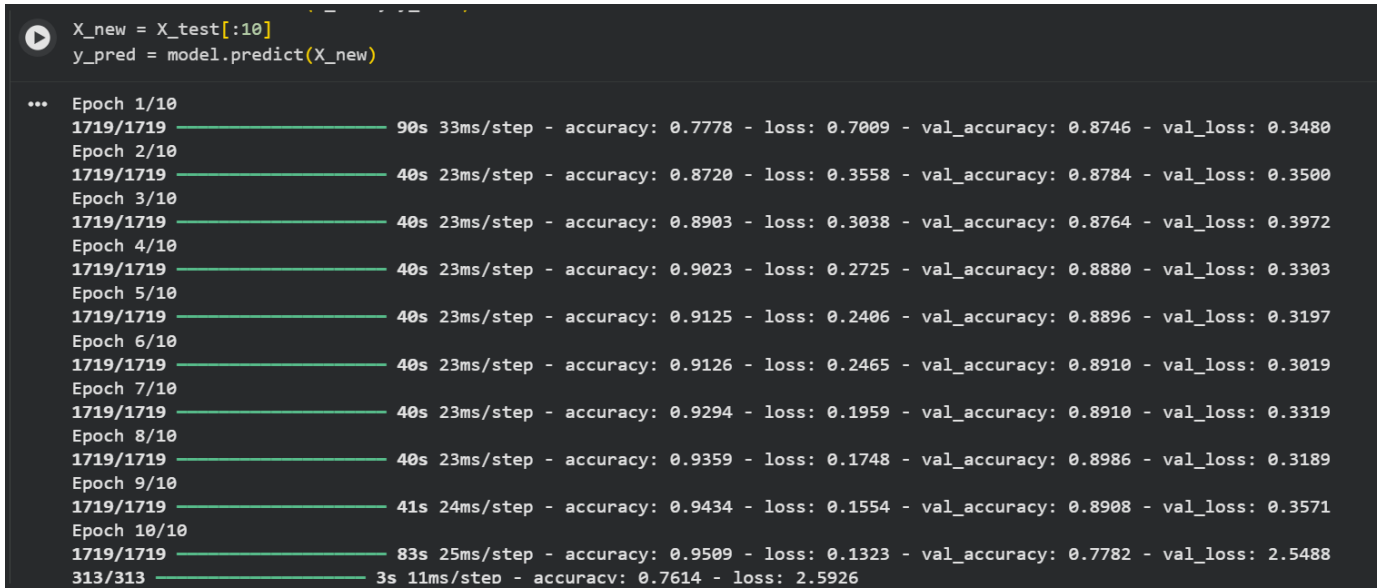


Fig. 9: Training and evaluation results each epoch.

IV. TASK 4- USE A PRE-TRAINED XCEPTION MODEL FOR TRANSFER LEARNING.

In this task, we implemented a convolutional neural network using a pre-trained Xception model from Keras Applications and applied it to a new classification task to TF Flowers dataset. Transfer learning allows leveraging features learned from large-scale datasets such as ImageNet and applying them to smaller, domain-specific datasets, improving training efficiency and performance.

A. Load a dataset (e.g., `tf_flowers`) and split it into training, validation, and testing datasets

The dataset was split into training, validation, and testing sets as follows.

- `train[:10%]` – first 10% of the dataset, used as the **test set**.
- `train[10%:25%]` – next 15% of the dataset, used as the **validation set**.
- `train[25%:]` – remaining 75% of the dataset, used as the **training set**.

B. Pre-process and resize the images.

Figure 10 shows sample images from the `tf_flowers` dataset after preprocessing. The images were resized to 224×224 pixels, which is the input size required by the Xception model.

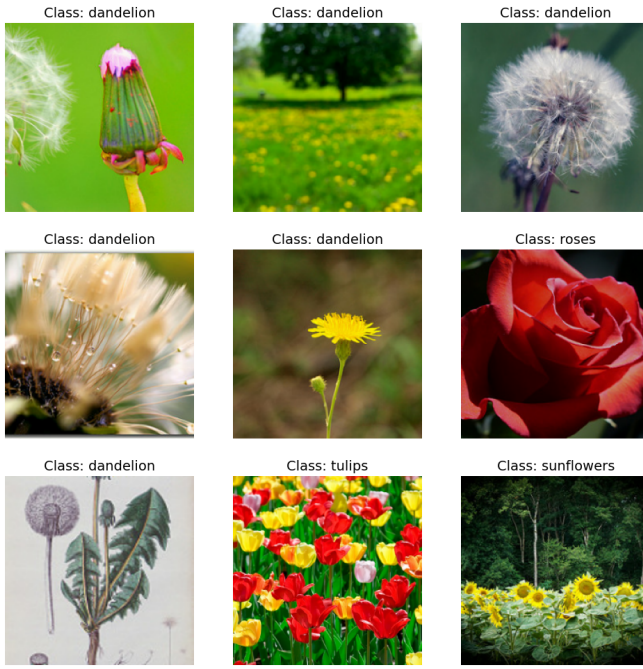


Fig. 10: first 9 images from the tf_flowers dataset.

C. Create the Keras model and freeze the weights of the pre-trained layers.

The Xception model, which is based on depthwise separable convolutions, was loaded with the following configuration:

- **Pre-trained Weights:** The model was initialised with imagenet weights, allowing it to reuse features learned on over one million images across 1000 classes.
- **Top Layer Exclusion:** The fully connected top layer (`include_top=False`) was removed to enable the addition of a custom classifier suitable for this tf_flowers dataset.
- **Global Average Pooling:** A `GlobalAveragePooling2D` layer was applied to the output feature maps of the base model. This reduces each feature map to a single value, converting the 2D spatial data into a 1D feature vector suitable for dense layers.
- **Custom Output Layer:** A dense layer with `softmax` activation was added as the output layer, with the number of units corresponding to the number of classes (`n_classes`) in the new dataset.

To preserve the knowledge learned on ImageNet and prevent it from being overwritten during training on this dataset, all layers of the pre-trained Xception base were frozen:

- Each layer in `base_model.layers` was set to `trainable = False`.
- Only the custom output layer is trainable, allowing the network to learn task-specific class mappings while preserving the pre-trained features.

D. Train the model and report the results.

Figure 11 and Figure 12 demonstrates the training and validation accuracy and loss for the transfer learning model using the pre-trained Xception architecture. The model was trained for 10 epochs, and the results indicate that transfer learning significantly improved performance compared to training a model from scratch on this dataset.

REFERENCES

- [1] GitHubRepo, "Authors github repository for laborations," Online: <https://github.com/Nilswik/DIL700/>, accessed: 2026-02-20.
- [2] GeeksForGeeks, "Residual networks (resnet) - deep learning," Online: <https://www.geeksforgeeks.org/deep-learning/residual-networks-resnet-deep-learning/>, accessed: 2026-02-20.


```
history = model.fit(train_set, validation_data=valid_set, epochs=10)

... Epoch 1/10
86/86 ————— 58s 477ms/step - accuracy: 0.8746 - loss: 0.3504 - val_accuracy: 0.8802 - val_loss: 0.4847
Epoch 2/10
86/86 ————— 32s 364ms/step - accuracy: 0.9864 - loss: 0.0466 - val_accuracy: 0.8966 - val_loss: 0.4196
Epoch 3/10
86/86 ————— 31s 358ms/step - accuracy: 0.9969 - loss: 0.0145 - val_accuracy: 0.8984 - val_loss: 0.3906
Epoch 4/10
86/86 ————— 32s 361ms/step - accuracy: 0.9961 - loss: 0.0134 - val_accuracy: 0.9020 - val_loss: 0.3966
Epoch 5/10
86/86 ————— 32s 364ms/step - accuracy: 0.9960 - loss: 0.0112 - val_accuracy: 0.9002 - val_loss: 0.4181
Epoch 6/10
86/86 ————— 32s 364ms/step - accuracy: 0.9981 - loss: 0.0037 - val_accuracy: 0.9129 - val_loss: 0.3661
Epoch 7/10
86/86 ————— 32s 361ms/step - accuracy: 0.9984 - loss: 0.0075 - val_accuracy: 0.9111 - val_loss: 0.4413
Epoch 8/10
86/86 ————— 32s 363ms/step - accuracy: 0.9990 - loss: 0.0030 - val_accuracy: 0.9074 - val_loss: 0.3803
Epoch 9/10
86/86 ————— 33s 369ms/step - accuracy: 0.9995 - loss: 0.0016 - val_accuracy: 0.9093 - val_loss: 0.3743
Epoch 10/10
86/86 ————— 32s 360ms/step - accuracy: 0.9993 - loss: 0.0017 - val_accuracy: 0.9002 - val_loss: 0.4090
```

Fig. 11: Training and validation accuracy for the transfer learning model using pre-trained Xception architecture.

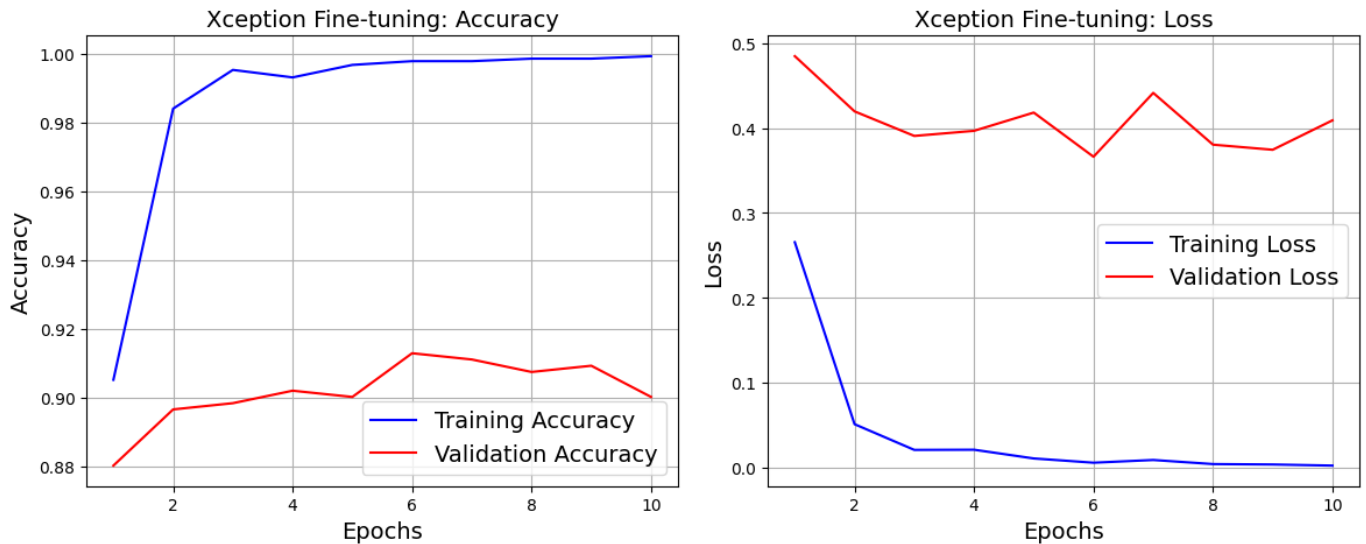


Fig. 12: Training and validation accuracy for the transfer learning model using pre-trained Xception architecture.